

Laboratorios Resueltos de Optimización y Análisis de Redes

Víctor Aceña Gil Antonio Alonso Ayuso

2026-06-16

Índice de Soluciones

Introducción	4
1 Laboratorio 1: introducción al modelado y cálculo simbólico con SymPy	5
2 Introducción	6
3 Configuración del entorno e instalación	7
4 Diferenciación simbólica y numérica en Python	8
4.1 Funciones univariantes	8
4.1.1 Enfoque numérico con Autograd	8
4.1.2 Enfoque simbólico con SymPy	8
4.2 Funciones multivariantes	9
4.2.1 Implementación con Autograd	9
4.2.2 Implementación con SymPy	10
4.3 Funciones paramétricas con SymPy	10
5 Estudio de curvatura y convexidad	11
6 Ejercicios de laboratorio propuestos	12
6.1 Ejercicio 1: gradiente y hessiana locales	12
6.1.1 1. Enfoque con Autograd	12
6.1.2 2. Enfoque con SymPy	13
6.2 Ejercicio 2: estudio de convexidad multivariable constante	13
6.3 Ejercicio 3: convexidad de funciones no cuadráticas (dominio factible)	14
6.4 Ejercicio 4: convexidad constante en 3D	16
7 Laboratorio 2: optimización no lineal con Python	18
8 Introducción	19
9 Fundamentos teóricos y herramientas de Python	20
9.1 Algoritmo del descenso de gradiente manual	20
9.1.1 Criterio de parada	20
9.2 Optimización de caja negra con <code>scipy.optimize.minimize</code>	20
9.2.1 Optimización sin restricciones: el método BFGS	21

9.2.2	Optimización con restricciones: el método SLSQP	21
9.3	Optimización Convexa Disciplinada con CVXPY	22
9.3.1	Reglas DCP Básicas	22
9.3.2	Obtención de Soluciones Duales (Multiplicadores de Lagrange)	23
9.4	Comparativa: SciPy vs CVXPY	23
10	Ejercicios de laboratorio propuestos	25
10.1	Ejercicio 1: descenso de gradiente manual	25
10.1.1	Tareas:	25
10.1.2	Código de resolución	26
10.1.3	Análisis de la convergencia	26
10.2	Ejercicio 2: optimización sin restricciones con SciPy	27
10.2.1	Tareas:	27
10.2.2	Código de resolución	27
10.2.3	Comparativa de rendimiento	28
10.3	Ejercicio 3: optimización restringida con CVXPY y SciPy (Teoría y Práctica) .	29
10.3.1	Tareas teóricas:	29
10.3.2	Tareas de código:	29
10.3.3	1. Resolución teórica (KKT)	30
10.3.4	2. Código de resolución en Python	31
10.3.5	3. Comparativa y análisis	32

Introducción

Este cuaderno recopila las **soluciones completas y detalladas** de los laboratorios prácticos de la asignatura **Optimización y Análisis de Redes** para el Grado en Matemáticas.

El objetivo de este manual es servir de recurso de referencia para comprobar la validez de los desarrollos algorítmicos y matemáticos propuestos, relacionando de forma sistemática la teoría de optimalidad y convexidad con las librerías científicas estándares de Python (**SymPy**, **Autograd**, **SciPy** y **CVXPY**).

Cada ejercicio incluye: 1. El enunciado original propuesto al alumno. 2. La resolución paso a paso detallada con código ejecutable en Python. 3. El análisis teórico de los resultados (como la comprobación matemática analítica mediante condiciones KKT o el análisis de autovalores).

Las soluciones se presentan en bloques colapsables para facilitar la autoevaluación interactiva.

1 Laboratorio 1: introducción al modelado y cálculo simbólico con SymPy

Optimización y Análisis de Redes

2 Introducción

El modelado y la resolución de problemas de optimización matemática requiere una sólida comprensión de las propiedades analíticas de las funciones, en especial de su **curvatura** y **convexidad**.

En este laboratorio práctico aprenderemos a utilizar Python para automatizar el análisis matemático local y global de funciones univariables y multivariantes. Trabajaremos con dos herramientas fundamentales y complementarias:

1. **SymPy**: Una biblioteca de computación simbólica que nos permite realizar cálculos exactos (analíticos) de derivadas, gradientes, matrices hessianas y resolver sistemas de ecuaciones algebraicas de forma exacta.
2. **Autograd**: Una biblioteca de diferenciación automática que calcula de manera eficiente y con precisión de máquina las derivadas de funciones definidas mediante código de Python ordinario (cálculo numérico).

3 Configuración del entorno e instalación

Para asegurar que dispones de las librerías necesarias en tu entorno de Python, puedes instalarlas ejecutando:

```
pip install autograd sympy numpy
```

A continuación, importamos los módulos que utilizaremos:

```
import numpy as np
from autograd import grad, hessian
import sympy as sp

# Configurar SymPy para que imprima las expresiones matemáticas con formato elegante
sp.init_printing(use_unicode=True)
```

4 Diferenciación simbólica y numérica en Python

4.1 Funciones univariantes

Consideremos la función real de una variable:

$$f(x) = 5x + 10$$

Deseamos calcular su derivada (gradiente) en el punto de referencia $x_0 = 1.0$.

4.1.1 Enfoque numérico con Autograd

Autograd realiza diferenciación automática a partir de la definición estándar de la función en Python:

```
def f_num(x):  
    return 5*x + 10  
  
# Obtener la función derivada de primer orden  
gradiente_num = grad(f_num)  
  
x0 = 1.0  
print("El gradiente numérico en x0 es:", gradiente_num(x0))
```

4.1.2 Enfoque simbólico con SymPy

SymPy construye un árbol de expresión analítica sobre una variable simbólica para realizar derivación formal:


```

# 1. Definir la variable simbólica
x = sp.symbols('x', real=True)

# 2. Definir la expresión matemática
f_sym = 5*x + 10

# 3. Calcular la derivada analítica
derivada_sym = sp.diff(f_sym, x)
print("Derivada analítica:", derivada_sym)

# 4. Evaluar la derivada en el punto x0
x0_val = 1.0
derivada_en_x0 = derivada_sym.subs(x, x0_val)
print("El gradiente simbólico evaluado en x0 es:", derivada_en_x0)

```

4.2 Funciones multivariables

Analicemos ahora una función lineal multivariable (un plano en $\mathbb{R}^2$):

$$f(x_1, x_2) = 5x_1 + 10x_2 + 4$$

Deseamos evaluar en el punto $\mathbf{x}_0 = (1, 2)^T$: - El vector gradiente: $\nabla f(\mathbf{x}_0)$ - La matriz hessiana: $\mathbf{H}_f(\mathbf{x}_0)$

4.2.1 Implementación con Autograd

```

def plano_num(x):
    return 5*x[0] + 10*x[1] + 4

x0 = np.array([1.0, 2.0])

# Obtener gradiente y hessiano
grad_plano = grad(plano_num)
hess_plano = hessian(plano_num)

print("Gradiente numérico en x0:", grad_plano(x0))
print("Hessiana numérica en x0:\n", hess_plano(x0))

```

4.2.2 Implementación con SymPy

```
# 1. Definir variables simbólicas
x1, x2 = sp.symbols('x1 x2', real=True)

# 2. Definir la función
f_plano_sym = 5*x1 + 10*x2 + 4

# 3. Calcular gradiente (como matriz columna de derivadas parciales)
grad_plano_sym = sp.Matrix([sp.diff(f_plano_sym, var) for var in (x1, x2)])

# 4. Calcular matriz hessiana
hess_plano_sym = sp.hessian(f_plano_sym, (x1, x2))

# 5. Evaluar en x0 = (1, 2)
eval_dict = {x1: 1.0, x2: 2.0}
print("Gradiente simbólico en x0:", grad_plano_sym.subs(eval_dict))
print("Hessiana simbólica en x0:\n", hess_plano_sym.subs(eval_dict))
```

4.3 Funciones paramétricas con SymPy

Una gran ventaja del cálculo simbólico es la capacidad de trabajar con parámetros sin asignarles valores de antemano. Consideremos el plano con coeficientes genéricos a y b :

$$f(x_1, x_2) = ax_1 + bx_2 + 4$$

```
# Definir variables y parámetros simbólicos
x1, x2, a, b = sp.symbols('x1 x2 a b', real=True)

f_param = a*x1 + b*x2 + 4

# Calcular gradiente y hessiano analíticos genéricos
grad_param = sp.Matrix([sp.diff(f_param, var) for var in (x1, x2)])
hess_param = sp.hessian(f_param, (x1, x2))

print("Gradiente analítico genérico:\n", grad_param)
print("\nHessiana analítica genérica:\n", hess_param)

# Evaluar en el punto x0 = (1, 2) con parámetros a = 5, b = 10
res_grad = grad_param.subs({x1: 1, x2: 2, a: 5, b: 10})
print("\nGradiente sustituyendo parámetros:", res_grad)
```

5 Estudio de curvatura y convexidad

Para evaluar si una función es convexa, cóncava o indefinida en un dominio, se estudia la definición de su matriz hessiana: - **Convexa**: La matriz hessiana es semidefinida positiva ($\mathbf{H}_f(\mathbf{x}) \succeq 0$) para todo $\mathbf{x}$. Sus autovalores son no negativos ($\lambda_i \geq 0$).

- **Estrictamente convexa**: La matriz hessiana es definida positiva ($\mathbf{H}_f(\mathbf{x}) \succ 0$) para todo $\mathbf{x}$. Sus autovalores son estrictamente positivos ($\lambda_i > 0$).
- **Cóncava**: La matriz hessiana es semidefinida negativa ($\mathbf{H}_f(\mathbf{x}) \preceq 0$) para todo $\mathbf{x}$. Sus autovalores son no positivos ($\lambda_i \leq 0$).

Podemos utilizar NumPy para hallar numéricamente los autovalores a partir del hessiano simbólico convertido:

```
# Definir una función auxiliar para obtener autovalores numéricos
def calcular_autovalores(matriz_sympy, punto_dict=None):
    if punto_dict:
        mat_eval = matriz_sympy.subs(punto_dict)
    else:
        mat_eval = matriz_sympy
    mat_np = np.array(mat_eval).astype(np.float64)
    return np.linalg.eigvals(mat_np)
```

6 Ejercicios de laboratorio propuestos

Resuelve de forma interactiva en tu cuaderno los siguientes ejercicios aplicando las herramientas que acabamos de describir.

6.1 Ejercicio 1: gradiente y hessiana locales

Dada la función de segundo grado:

$$f(\mathbf{x}) = 2x_1 + 6x_2 - 2x_1^2 - 3x_2^2 + 4x_1x_2$$

Calcula su vector gradiente y su matriz hessiana en el punto de origen $\mathbf{x}_0 = (0,0)^T$ de dos maneras:

1. Empleando **Autograd**.
2. Empleando **SymPy**.

```
# Escribe tu solución aquí:  
# ...
```

[Pincha aquí para ver la solución](#)

6.1.1 1. Enfoque con Autograd

```
import numpy as np  
from autograd import grad, hessian  
  
# Definimos la función en Python  
def f_num(x):  
    return 2*x[0] + 6*x[1] - 2*x[0]**2 - 3*x[1]**2 + 4*x[0]*x[1]  
  
# Punto de evaluación  
x0 = np.array([0.0, 0.0])
```

```
# Obtenemos las funciones para gradiente y hessiana
grad_f = grad(f_num)
hess_f = hessian(f_num)

print("Gradiente (Autograd):", grad_f(x0))
print("Hessiana (Autograd):\n", hess_f(x0))
```

6.1.2 2. Enfoque con SymPy

```
import sympy as sp

# Definimos variables simbólicas
x1, x2 = sp.symbols('x1 x2', real=True)

# Definimos la función
f_sym = 2*x1 + 6*x2 - 2*x1**2 - 3*x2**2 + 4*x1*x2

# Calculamos gradiente (matriz columna) y hessiana
grad_sym = sp.Matrix([sp.diff(f_sym, var) for var in (x1, x2)])
hess_sym = sp.hessian(f_sym, (x1, x2))

# Punto de evaluación en formato diccionario
eval_dict = {x1: 0.0, x2: 0.0}

print("Gradiente (SymPy):", grad_sym.subs(eval_dict))
print("Hessiana (SymPy):\n", hess_sym.subs(eval_dict))
```

El resultado esperado para el gradiente en el origen es $[2., 6.]$ y para la matriz hessiana es:

$$\mathbf{H}_f(0,0) = \begin{pmatrix} -4 & 4 \\ 4 & -6 \end{pmatrix}$$

6.2 Ejercicio 2: estudio de convexidad multivariable constante

Estudia analíticamente la convexidad de la siguiente función cuadrática en todo su dominio:

$$f(\mathbf{x}) = x_1x_2 + 2x_1^2 + x_2^2 - 6x_1x_3$$

Para ello:

1. Define simbólicamente la función en SymPy.
2. Obtén la matriz hessiana simbólica.
3. Convierte la matriz a NumPy y calcula sus autovalores.
4. Con base en el signo de los autovalores, concluye si la función es convexa, cóncava o indefinida.

```
# Escribe tu solución aquí:
# ...
```

Pincha aquí para ver la solución

```
import sympy as sp
import numpy as np

# 1. Definir variables simbólicas
x1, x2, x3 = sp.symbols('x1 x2 x3', real=True)

# Definir la expresión matemática
f_sym = x1*x2 + 2*x1**2 + x2**2 - 6*x1*x3

# 2. Obtener la matriz hessiana simbólica
hess_sym = sp.hessian(f_sym, (x1, x2, x3))
print("Hessiana simbólica:\n", hess_sym)

# 3. Convertir a NumPy y calcular autovalores
hess_np = np.array(hess_sym).astype(np.float64)
autovalores = np.linalg.eigvals(hess_np)
print("\nAutovalores de la Hessiana:", autovalores)
```

Análisis de resultados: Los autovalores calculados de la hessiana son aproximadamente $\lambda_1 \approx 8.427$, $\lambda_2 \approx -4.379$ y $\lambda_3 \approx 1.951$. Al existir autovalores de signo opuesto (tanto positivos como negativos), la matriz hessiana es **indefinida** y, al ser constante en todo el dominio, concluimos que la función **no es convexa ni cóncava** en $\mathbb{R}^3$.

6.3 Ejercicio 3: convexidad de funciones no cuadráticas (dominio factible)

Estudia la convexidad de la función:

$$f(\mathbf{x}) = 10 - 2(x_2 - x_1^2)^2$$

1. Calcula la matriz hessiana simbólica en términos de las variables x_1 y x_2 .
2. Dado que los elementos de la matriz dependen de las variables, la convexidad varía en distintas regiones del espacio. Encuentra analíticamente la región o conjunto de puntos (x_1, x_2) donde la función es **cóncava** (es decir, donde el menor principal de primer orden y el determinante alternan de signo: $M_1 \leq 0$ y $M_2 \geq 0$).
3. Utiliza la función `sp.solve` para hallar la región correspondiente de la variable x_2 en función de x_1 .

```
# Escribe tu solución aquí:
# ...
```

Pincha aquí para ver la solución

```
import sympy as sp

# Definir variables simbólicas
x1, x2 = sp.symbols('x1 x2', real=True)

# Definir la función
f_sym = 10 - 2*(x2 - x1**2)**2

# 1. Calcular la hessiana
hess_sym = sp.hessian(f_sym, (x1, x2))
print("Matriz Hessiana:\n", hess_sym)

# 2. Menores principales
M1 = hess_sym[0, 0]          # menor principal de 1er orden
M2 = hess_sym.det()          # menor principal de 2º orden (determinante)

print("\nM1 (H_11):", M1)
print("M2 (det H):", M2)

# 3. Resolver analíticamente la región M1 <= 0 y M2 >= 0
# M1 <= 0 => 8*x2 - 24*x1^2 <= 0 => x2 <= 3*x1^2
# M2 >= 0 => -32*x2 + 32*x1^2 >= 0 => x2 <= x1^2
# Intersecando ambas regiones, predomina la condición más restrictiva: x2 <= x1^2.
```

Análisis de resultados: La hessiana resultante es:

$$\mathbf{H}_f(\mathbf{x}) = \begin{pmatrix} -24x_1^2 + 8x_2 & 8x_1 \\ 8x_1 & -4 \end{pmatrix}$$

Los menores principales del hessiano son: - $M_1 = 8x_2 - 24x_1^2 \leq 0$ - $M_2 = \det(\mathbf{H}_f) = -32x_2 + 32x_1^2 \geq 0$

Resolviendo simultáneamente ambas inecuaciones, se obtiene que la función es cóncava (semidefinida negativa) en la región parabólica delimitada por:

$$x_2 \leq x_1^2$$

6.4 Ejercicio 4: convexidad constante en 3D

Determina si la función:

$$f(\mathbf{x}) = x^2 + 3y^2 + z^2 + xy - 4z - 5x + 14$$

es estrictamente convexa en su dominio calculando los autovalores de su matriz hessiana con SymPy y NumPy.

```
# Escribe tu solución aquí:  
# ...
```

Pincha aquí para ver la solución

```
import sympy as sp  
import numpy as np  
  
# Definir variables simbólicas  
x, y, z = sp.symbols('x y z', real=True)  
  
# Definir la función  
f_sym = x**2 + 3*y**2 + z**2 + x*y - 4*z - 5*x + 14  
  
# Calcular la hessiana  
hess_sym = sp.hessian(f_sym, (x, y, z))  
print("Hessiana simbólica:\n", hess_sym)  
  
# Calcular autovalores numéricos  
hess_np = np.array(hess_sym).astype(np.float64)  
autovalores = np.linalg.eigvals(hess_np)  
print("\nAutovalores de la Hessiana:", autovalores)
```


Análisis de resultados: La matriz hessiana es constante en todo el dominio:

$$\mathbf{H}_f = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 6 & 0 \\ 0 & 0 & 2 \end{pmatrix}$$

Los autovalores calculados de la hessiana son aproximadamente $\lambda_1 \approx 6.236$, $\lambda_2 \approx 1.764$ y $\lambda_3 = 2.0$. Dado que todos los autovalores son estrictamente mayores que cero, la matriz hessiana es **definida positiva** en todo $\mathbb{R}^3$, lo que demuestra que la función es **estrictamente convexa** de manera global.

7 Laboratorio 2: optimización no lineal con Python

Optimización y Análisis de Redes

8 Introducción

La optimización no lineal es una de las disciplinas matemáticas con mayor impacto en la práctica industrial, científica y tecnológica. Mientras que algunos problemas de juguete pueden resolverse de manera analítica (por ejemplo, igualando el gradiente a cero o aplicando las condiciones de Karush-Kuhn-Tucker), en la práctica la complejidad de las funciones y el número de variables nos obligan a recurrir a **algoritmos numéricos iterativos**.

El objetivo de este laboratorio es aprender a relacionar los fundamentos teóricos de la optimización matemática con el uso de herramientas profesionales en Python. A lo largo de la sesión aprenderemos a:

1. Diseñar e implementar desde cero el algoritmo clásico de **descenso de gradiente**, analizando la influencia de la tasa de aprendizaje y el comportamiento iterativo del algoritmo.
 2. Utilizar la biblioteca científica estándar **scipy.optimize** (en concreto, su función central **minimize**), entendiendo cómo configurar solvers libres de restricciones (como **BFGS**) y solvers con restricciones generales (como **SLSQP**).
 3. Utilizar la biblioteca especializada **CVXPY** para resolver problemas de optimización convexa disciplinada, explorando sus ventajas en cuanto a declaración matemática y obtención directa de las **soluciones duales (multiplicadores de Lagrange)**.
-

9 Fundamentos teóricos y herramientas de Python

9.1 Algoritmo del descenso de gradiente manual

El método del **descenso de gradiente** (o *gradient descent*) es el algoritmo de búsqueda local de primer orden más elemental. Para resolver el problema de minimización sin restricciones:

$$\min_{x \in \mathbb{R}^n} f(x)$$

el algoritmo genera una secuencia de puntos $\{x^{(k)}\}$ partiendo de una estimación inicial $x^{(0)}$. Dado que el gradiente $\nabla f(x)$ apunta en la dirección de máximo crecimiento local, nos movemos en la dirección opuesta $-\nabla f(x)$ para disminuir el valor de la función:

$$x^{(k+1)} = x^{(k)} - \alpha \nabla f(x^{(k)})$$

donde: * $x^{(k)}$ es el punto en la iteración k . * $\alpha > 0$ es la **tasa de aprendizaje** (*learning rate*) o tamaño de paso. * Si α es demasiado pequeño, la convergencia será extremadamente lenta. * Si α es demasiado grande, el algoritmo puede oscilar e incluso divergir. * $\nabla f(x^{(k)})$ es el gradiente de la función en el punto actual.

9.1.1 Criterio de parada

En una implementación real, la iteración se detiene cuando se cumple alguna de las siguientes condiciones: 1. Se alcanza un número máximo de iteraciones ($k \geq K_{\max}$). 2. El gradiente es prácticamente cero, lo que indica que estamos en un punto estacionario: $\|\nabla f(x^{(k)})\|_2 < \epsilon$, donde $\epsilon > 0$ es una tolerancia (por ejemplo, 10^{-6}).

9.2 Optimización de caja negra con `scipy.optimize.minimize`

El módulo `scipy.optimize` es el estándar en Python para optimización numérica general. Su función estrella es `minimize`, que actúa como interfaz común para diversos métodos numéricos.

9.2.1 Optimización sin restricciones: el método BFGS

El algoritmo **BFGS** (Broyden-Fletcher-Goldfarb-Shanno) es un método de tipo **Quasi-Newton**. A diferencia del método de Newton clásico, que requiere calcular la matriz hessiana inversa ($H^{-1}(x)$) en cada iteración (lo cual es costoso computacionalmente), BFGS aproxima dicha matriz de forma iterativa utilizando únicamente información de los gradientes sucesivos.

La llamada básica a `minimize` tiene el siguiente aspecto:

```
from scipy.optimize import minimize

res = minimize(fun, x0, method='BFGS', jac=grad_fun)
```

Donde: * `fun`: Función objetivo que recibe un array de NumPy y devuelve un escalar. * `x0`: Punto inicial (vector o lista de Python). * `method='BFGS'`: Especifica que usaremos el algoritmo Quasi-Newton BFGS. * `jac`: Función que calcula el gradiente (Jacobiano). * **Importante**: Si `jac=None` (o no se proporciona), `minimize` estimará el gradiente numéricamente mediante **diferencias finitas**. Esto requiere evaluar la función objetivo múltiples veces por cada paso (al menos $n + 1$ evaluaciones, donde n es la dimensión), lo que incrementa el coste computacional y puede introducir imprecisiones numéricas.

El resultado devuelto (`res`) es un objeto `OptimizeResult` con los siguientes campos clave: * `res.x`: El punto óptimo encontrado. * `res.fun`: El valor de la función en el óptimo. * `res.success`: Booleano que indica si el solver convergió con éxito. * `res.message`: Mensaje descriptivo de la terminación. * `res.nfev`: Número de evaluaciones de la función objetivo. * `res.njev`: Número de evaluaciones del gradiente (Jacobiano).

9.2.2 Optimización con restricciones: el método SLSQP

Cuando existen restricciones analíticas, el método por defecto de SciPy es **SLSQP** (Sequential Least Squares Programming). SLSQP formula y resuelve aproximaciones cuadráticas del problema original sujeto a la linealización de las restricciones en cada paso.

Para definir restricciones en SciPy se utilizan diccionarios dentro de una lista: 1. **Inecuaciones**: En SciPy se asume que las restricciones de desigualdad están formuladas como $g(x) \geq 0$. Si tu problema teórico tiene la forma estándar $h(x) \leq c$, debes reescribirla como $c - h(x) \geq 0$. 2. **Ecuaciones**: Restricciones de igualdad de la forma $e(x) = 0$.

Ejemplo de sintaxis:

```
# Restricción:  $x_1 + 2x_2 \geq 4 \implies x_1 + 2x_2 - 4 \geq 0$ 
def restriccion_lineal(x):
    return x[0] + 2*x[1] - 4

# Lista de restricciones
restricciones = [
    {'type': 'ineq', 'fun': restriccion_lineal}
]

# Límites de no negatividad:  $x_1 \geq 0, x_2 \geq 0$ 
cotas = [(0, None), (0, None)]

res = minimize(fun, x0, method='SLSQP', bounds=cotas, constraints=restricciones)
```

9.3 Optimización Convexa Disciplinada con CVXPY

CVXPY es una biblioteca de modelado declarativo en Python diseñada específicamente para **optimización convexa**. A diferencia de SciPy (que trata el problema como una “caja negra” numérica), CVXPY analiza la estructura matemática del problema y comprueba si cumple las reglas de la **Programación Convexa Disciplinada (DCP)**.

9.3.1 Reglas DCP Básicas

Para que CVXPY acepte y resuelva un problema, este debe cumplir una serie de reglas de composición geométrica que aseguran que el problema es verdaderamente convexo:

- * **Función Objetivo:** Si minimizamos, el objetivo debe ser convexo (p. ej., x^2 , $\|x\|_2$, $\sum x^2$). Si maximizamos, debe ser cóncavo (p. ej., $\sqrt{x}$, $\log x$).
- * **Restricciones de desigualdad:** Deben tener la forma **convexo** $\leq$ **concavo** o **concavo** $\geq$ **convexo**.
- * **Restricciones de igualdad:** Deben ser afines (lineales) de la forma **lineal** $==$ **lineal**.
- * **Operaciones prohibidas:** No se permite multiplicar variables entre sí (p. ej. $x * y$ o $x[0] * x[1]$), ya que esto rompe las reglas de convexidad generalizadas de DCP. Para representar términos cuadráticos como $x^T P x$, se deben usar funciones específicas como `cp.quad_form(x, P)` o `cp.sum_squares(x)`.

9.3.2 Obtención de Soluciones Duales (Multiplicadores de Lagrange)

En la teoría de la optimización, las **variables duales** (o multiplicadores de Lagrange/KKT asociados a las restricciones) nos indican la sensibilidad del valor óptimo de la función objetivo ante pequeñas variaciones en los límites de las restricciones (también llamados *precios sombra*).

CVXPY recupera de forma nativa e inmediata estas variables duales:

```
import cvxpy as cp

# 1. Definir variables
x = cp.Variable(2, nonneg=True) # nonneg=True añade la restricción x >= 0

# 2. Definir objetivo
objetivo = cp.Minimize(0.5 * cp.sum_squares(x))

# 3. Definir la restricción y asignarle un objeto
restriccion = x[0] + 2*x[1] >= 4
restricciones = [restriccion]

# 4. Resolver
problema = cp.Problem(objetivo, restricciones)
problema.solve()

# 5. Obtener los multiplicadores de Lagrange (dualidad)
valor_dual = restriccion.dual_value
print("Multiplicador de Lagrange de la restricción:", valor_dual)
```

9.4 Comparativa: SciPy vs CVXPY

Característica	scipy.optimize.minimize	CVXPY
Paradigma	Numérico procedural (caja negra).	Declarativo (lenguaje de modelado algebraico).
Garantías	Encuentra óptimos locales en problemas no convexos.	Garantiza óptimos globales en problemas convexos.
Validación	No comprueba si el problema es resoluble o convexo.	Valida estrictamente bajo las reglas DCP.

Característica	<code>scipy.optimize.minimize</code>	CVXPY
Derivadas	Requiere el gradiente de forma explícita o diferencias finitas.	Realiza diferenciación simbólica/automática interna.
Acceso a Dualidad	Complejo y dependiente del solver interno.	Directo mediante la propiedad <code>.dual_value</code> .

10 Ejercicios de laboratorio propuestos

10.1 Ejercicio 1: descenso de gradiente manual

Implementa un script en Python para resolver mediante descenso de gradiente manual la minimización de la función cuadrática:

$$f(x, y) = x^2 + 2y^2$$

10.1.1 Tareas:

1. Define la función objetivo $f(x, y)$ y su gradiente analítico $\nabla f(x, y) = (2x, 4y)^T$ en Python.
2. Implementa el bucle iterativo partiendo de $x^{(0)} = (2, 2)^T$ con una tasa de aprendizaje constante $\alpha = 0.1$ durante un total de $K = 10$ iteraciones.
3. Imprime en cada iteración: el número de iteración, el punto $(x^{(k)}, y^{(k)})$ y el valor de la función objetivo $f(x^{(k)}, y^{(k)})$.
4. Reflexiona: ¿El algoritmo ha llegado al mínimo global exacto $(0, 0)$ en 10 iteraciones? ¿Por qué?

```
# Tu código aquí:
import numpy as np

def f(x):
    # x es un array de dimensión 2
    return x[0]**2 + 2*x[1]**2

def grad_f(x):
    return np.array([2*x[0], 4*x[1]])

# Punto inicial
x_k = np.array([2.0, 2.0])
alpha = 0.1
max_iter = 10

# Bucle de optimización
# ...
```

Pincha aquí para ver la solución

10.1.2 Código de resolución

```
import numpy as np

def f(x):
    return x[0]**2 + 2*x[1]**2

def grad_f(x):
    return np.array([2.0 * x[0], 4.0 * x[1]])

# Parámetros del algoritmo
x_k = np.array([2.0, 2.0])
alpha = 0.1
max_iter = 10

print("Trazado de iteraciones:")
for k in range(max_iter + 1):
    val = f(x_k)
    print(f"Iteración {k:2d}: x = [{x_k[0]:.6f}, {x_k[1]:.6f}], f(x) = {val:.8f}")
    if k < max_iter:
        x_k = x_k - alpha * grad_f(x_k)
```

10.1.3 Análisis de la convergencia

El resultado de la ejecución muestra que en la iteración 10 el punto es aproximadamente $x^{(10)} \approx (0.2147, 0.0121)^T$ con un valor objetivo de $f(x^{(10)}) \approx 0.0464$.

Reflexión teórica: 1. **Convergencia lineal:** El método de descenso de gradiente ordinario tiene una velocidad de convergencia lineal. Esto implica que el error disminuye en una proporción constante en cada paso, requiriendo teóricamente un número infinito de iteraciones ($k \rightarrow \infty$) para alcanzar el mínimo global exacto $(0, 0)$. 2. **Efecto de la curvatura:** La tasa de convergencia analítica para cada coordenada sigue las progresiones geométricas: $x^{(k)} = x^{(0)}(1 - 2\alpha)^k = 2(0.8)^k$ * $y^{(k)} = y^{(0)}(1 - 4\alpha)^k = 2(0.6)^k$ Dado que los autovalores de la matriz hessiana son distintos (2 y 4), la curvatura es asimétrica (elipse). La coordenada y (curvatura más pronunciada) converge más rápido hacia cero que la coordenada x , provocando una trayectoria de aproximación no homogénea. En 10 iteraciones, $x^{(10)} = 2(0.8)^{10} \approx 0.2147$, lo que impide alcanzar el óptimo exacto de forma finita.

10.2 Ejercicio 2: optimización sin restricciones con SciPy

Resuelve la misma optimización sin restricciones del Ejercicio 1:

$$\min \quad f(x, y) = x^2 + 2y^2$$

utilizando la función `minimize` de `scipy.optimize`.

10.2.1 Tareas:

1. Realiza la optimización de dos maneras distintas:
 - **Caso A:** Sin proporcionar el gradiente (dejando que SciPy calcule las diferencias finitas).
 - **Caso B:** Proporcionando el gradiente analítico a través del parámetro `jac`.
2. Para ambos casos, utiliza el método `'BFGS'` e inicializa en $x^{(0)} = (2, 2)^T$.
3. Imprime el informe completo de optimización de ambos casos. Compare los valores finales de `nfev` (evaluaciones de la función) y `njev` (evaluaciones del gradiente) de cada caso y con los de tu método manual. ¿Qué diferencias observas?

```
# Tu código aquí:
from scipy.optimize import minimize

# Caso A: sin gradiente
# ...

# Caso B: con gradiente analítico
# ...
```

[Pincha aquí para ver la solución](#)

10.2.2 Código de resolución

```
import numpy as np
from scipy.optimize import minimize

def f(x):
    return x[0]**2 + 2*x[1]**2
```

```
def grad_f(x):
    return np.array([2.0 * x[0], 4.0 * x[1]])

x0 = np.array([2.0, 2.0])

# Caso A: Estimación numérica del gradiente
res_a = minimize(f, x0, method='BFGS')
print("--- CASO A (Sin gradiente analítico) ---")
print(res_a)

# Caso B: Gradiente analítico suministrado
res_b = minimize(f, x0, method='BFGS', jac=grad_f)
print("\n--- CASO B (Con gradiente analítico) ---")
print(res_b)
```

10.2.3 Comparativa de rendimiento

1. **Precisión:** Ambos casos convergen al mínimo global con una precisión extremadamente alta (valor objetivo $f(x^*) \approx 2.6 \times 10^{-12}$), localizando el punto óptimo prácticamente en $(0, 0)$ en comparación con el descenso manual que quedó en 0.0464.
2. **Número de iteraciones (nit):** Ambos casos requieren exactamente 7 iteraciones del método Quasi-Newton BFGS para satisfacer la tolerancia de convergencia.
3. **Evaluación de funciones (nfev y njev):**
 - **Caso A (Diferencias finitas):** Requiere **24 evaluaciones de la función objetivo** y **8 evaluaciones del gradiente** (estimado numéricamente). Para estimar cada componente del gradiente en $\mathbb{R}^2$ mediante diferencias finitas, el algoritmo necesita evaluar la función original en puntos perturbados vecinos, disparando el contador nfev.
 - **Caso B (Analítico):** Requiere únicamente **8 evaluaciones de la función objetivo** y **8 evaluaciones de la función del gradiente**. Al proporcionar grad_f analíticamente, el solver no requiere perturbar las variables para hallar pendientes, reduciendo drásticamente el coste de computación a 1 evaluación por paso.
4. **Comparación con el descenso manual:** Mientras que el descenso manual realiza un avance puramente lineal de 10 pasos fijos sin alcanzar el óptimo exacto, BFGS usa la aproximación de la curvatura (matriz hessiana inversa aproximada `hess_inv`) para dar pasos inteligentes de longitud variable (superlineales), alcanzando la precisión de máquina en 7 pasos.

10.3 Ejercicio 3: optimización restringida con CVXPY y SciPy (Teoría y Práctica)

Consideremos el siguiente problema de programación cuadrática con restricciones lineales y de no negatividad:

$$\begin{array}{ll} \underset{x_1, x_2}{\text{minimizar}} & f(x_1, x_2) = \frac{1}{2}(x_1^2 + x_2^2) \\ \text{sujeto a} & x_1 + 2x_2 \geq 4 \\ & x_1 \geq 0, x_2 \geq 0 \end{array}$$

10.3.1 Tareas teóricas:

1. Escribe el Lagrangiano del problema utilizando los multiplicadores de Lagrange μ (para la restricción principal) y λ_1, λ_2 (para las de no negatividad).
2. Plantea las condiciones de optimalidad de Karush-Kuhn-Tucker (KKT).
3. Resuelve analíticamente el sistema KKT para obtener la solución óptima primal (x_1^*, x_2^*) y el multiplicador de Lagrange óptimo de la restricción principal (μ^*) .

10.3.2 Tareas de código:

1. Resuelve el problema utilizando **CVXPY**. Recupera el punto óptimo primal, el valor objetivo y el multiplicador de Lagrange asociado a la restricción $x_1 + 2x_2 \geq 4$ empleando su propiedad `.dual_value`.
2. Resuelve el problema utilizando **scipy.optimize.minimize** con el método 'SLSQP'. Define correctamente las cotas de no negatividad (**bounds**) y la restricción principal como un diccionario (recuerda que SciPy asume que las restricciones de desigualdad son de la forma $g(x) \geq 0$).
3. Compara los resultados de ambos códigos con tu cálculo analítico. ¿Coinciden el punto óptimo y el multiplicador de Lagrange obtenido? Explica los resultados.

```
# Tu código aquí:

# 1. Resolución con CVXPY
# ...

# 2. Resolución con SciPy SLSQP
# ...
```

Pincha aquí para ver la solución

10.3.3 1. Resolución teórica (KKT)

Paso 1: Formulación del Lagrangiano Expresamos las restricciones en formato estándar de menor o igual a cero: * $-x_1 - 2x_2 + 4 \leq 0$ (multiplicador $\mu \geq 0$) * $-x_1 \leq 0$ (multiplicador $\lambda_1 \geq 0$) * $-x_2 \leq 0$ (multiplicador $\lambda_2 \geq 0$)

El Lagrangiano $L(x_1, x_2, \mu, \lambda_1, \lambda_2)$ es:

$$L(x_1, x_2, \mu, \lambda_1, \lambda_2) = \frac{1}{2}(x_1^2 + x_2^2) - \mu(x_1 + 2x_2 - 4) - \lambda_1 x_1 - \lambda_2 x_2$$

Paso 2: Planteamiento de las Condiciones KKT 1. Estacionariedad (anulación del gradiente):

$$\frac{\partial L}{\partial x_1} = x_1 - \mu - \lambda_1 = 0 \implies x_1 = \mu + \lambda_1$$

$$\frac{\partial L}{\partial x_2} = x_2 - 2\mu - \lambda_2 = 0 \implies x_2 = 2\mu + \lambda_2$$

2. Viabilidad Primal:

$$x_1 + 2x_2 \geq 4, \quad x_1 \geq 0, \quad x_2 \geq 0$$

3. Viabilidad Dual:

$$\mu \geq 0, \quad \lambda_1 \geq 0, \quad \lambda_2 \geq 0$$

4. Holgura Complementaria:

$$\mu(x_1 + 2x_2 - 4) = 0$$

$$\lambda_1 x_1 = 0$$

$$\lambda_2 x_2 = 0$$

Paso 3: Resolución analítica del sistema * El origen $(0,0)^T$ no cumple la restricción principal, por lo que la restricción $x_1 + 2x_2 \geq 4$ debe ser activa en el óptimo. Así, $\mu > 0$ y $x_1 + 2x_2 = 4$. * Asumimos razonablemente que las variables óptimas serán estrictamente positivas ($x_1 > 0$ y $x_2 > 0$). Por holgura complementaria, esto implica que las restricciones de no negatividad no están activas y sus multiplicadores son nulos: $\lambda_1 = 0$ y $\lambda_2 = 0$. * Sustituyendo en las condiciones de estacionariedad:

$$x_1 = \mu$$

$$x_2 = 2\mu$$

* Sustituimos estas expresiones en la restricción activa:

$$\mu + 2(2\mu) = 4 \implies 5\mu = 4 \implies \mu^* = 0.8$$

* Por tanto, los valores primales óptimos son:

$$x_1^* = 0.8, \quad x_2^* = 1.6$$

* Comprobamos viabilidad primal y dual: $x_1^* = 0.8 \geq 0$, $x_2^* = 1.6 \geq 0$, y $0.8 + 2(1.6) = 4 \geq 4$. Además, los multiplicadores $\mu^* = 0.8 \geq 0$ y $\lambda_1^* = \lambda_2^* = 0 \geq 0$ son válidos. Las condiciones KKT se cumplen por completo. * El valor mínimo de la función objetivo es:

$$f(x_1^*, x_2^*) = \frac{1}{2}(0.8^2 + 1.6^2) = \frac{1}{2}(0.64 + 2.56) = 1.6$$

10.3.4 2. Código de resolución en Python

```
import cvxpy as cp
import numpy as np
from scipy.optimize import minimize

# =====
# 1. RESOLUCIÓN CON CVXPY
# =====
x = cp.Variable(2, nonneg=True)
objetivo_cvx = cp.Minimize(0.5 * cp.sum_squares(x))
restriccion_cvx = x[0] + 2 * x[1] >= 4
restricciones_cvx = [restriccion_cvx]

problema = cp.Problem(objetivo_cvx, restricciones_cvx)
problema.solve()

print("--- RESULTADOS CVXPY ---")
print(f"Punto óptimo primal: {x.value}")
print(f"Valor objetivo mínimo: {problema.value:.6f}")
print(f"Multiplicador de Lagrange (dual): {restriccion_cvx.dual_value:.6f}")

# =====
# 2. RESOLUCIÓN CON SCIPY (SLSQP)
# =====
def f_obj(x):
    return 0.5 * (x[0]**2 + x[1]**2)
```

```

def grad_obj(x):
    return np.array([x[0], x[1]])

# Restricción principal:  $x_1 + 2x_2 \geq 4 \implies x_1 + 2x_2 - 4 \geq 0$ 
def restriccion_principal(x):
    return x[0] + 2.0 * x[1] - 4.0

restricciones_scipy = [
    {'type': 'ineq', 'fun': restriccion_principal}
]

# Cotas de no negatividad
limites = [(0, None), (0, None)]
x0 = np.array([2.0, 2.0])

res_scipy = minimize(f_obj, x0, method='SLSQP', bounds=limites, constraints=restricciones_scipy)

print("\n--- RESULTADOS SCIPY (SLSQP) ---")
print(f"Punto óptimo primal: {res_scipy.x}")
print(f"Valor objetivo mínimo: {res_scipy.fun:.6f}")
# SLSQP devuelve las variables duales bajo el campo "multipliers" de forma nativa en su Optimizer
if hasattr(res_scipy, 'multipliers'):
    print(f"Multiplicadores de Lagrange (SciPy): {res_scipy.multipliers}")

```

10.3.5 3. Comparativa y análisis

- **Coincidencia de valores:** Ambos solvers coinciden exactamente con la solución matemática analítica de KKT. Encuentran el punto primal $x^* = (0.8, 1.6)^T$, el valor óptimo de la función $f(x^*) = 1.6$ y el multiplicador de Lagrange de la restricción principal $\mu^* = 0.8$.
- **Gestión de variables duales:**
 - **CVXPY:** Ofrece la forma más limpia y robusta de recuperar la información dual. Asignando la inecuación a una variable (`restriccion_cvx`), podemos acceder de manera declarativa y directa a `.dual_value` una vez resuelto el problema.
 - **SciPy (SLSQP):** Devuelve el multiplicador de Lagrange óptimo de las restricciones en el campo `res.multipliers` (o a veces mediante aproximaciones del Jacobiano final). Sin embargo, este comportamiento depende del solver utilizado (otros métodos de `minimize` no exponen de manera explícita o cómoda las variables duales), lo que muestra que SciPy se enfoca en optimización primal de caja negra numérica general, mientras que CVXPY está explícitamente diseñado para explotar la estructura dual de los problemas convexos.